# Testing the chip RTL on a PYNQ-Z2 (or PYNQ-Z1) FPGA

The FPGA build uses **exactly the same Verilog as the chip** (`src/`), plus a small harness (`fpga/`):

- `fpga/rtl/qspi_pmod_emu.v`: the QSPI flash + 2 PSRAMs emulated in block RAM (64 KB each), with the flash preloaded with the demo firmware. You don't need the QSPI Pmod.
- `fpga/rtl/uart_watch.v`: watches the chip's UART TX and lights the RGB LEDs on `Hello, TT!`, `SELFTEST PASS` or `FAIL`, so you can see a pass without a PC.
- `fpga/pynq_z2/top_pynq_z2.v` + `pynq_z2.xdc`: 125 MHz -> 64 MHz MMCM, reset, and the pin mapping to buttons, switches, LEDs and Pmods.

The harness was checked in simulation (`make -C fpga sim`). The chip prints `Hello, TT!` and `SELFTEST PASS`, and the LED chaser runs.

## 1. Build the bitstream

Vivado 2025.1 is installed on this laptop (`C:\Xilinx\2025.1`). From Windows PowerShell:

```
powershell -ExecutionPolicy Bypass -File \\wsl.localhost\Ubuntu-22.04\home\santhosh\tt-sky26d-rvsoc\fpga\pynq_z2\build_on_windows.ps1
```

This builds the firmware in WSL, converts it for the block RAM, runs Vivado in batch mode and writes `fpga/build/rvsoc_pynq_z2.bit` with `utilization.rpt` and `timing_summary.rpt`. Add `-Program` to also load it over JTAG.

From WSL with Vivado on the PATH: `make -C fpga bit`, then `make -C fpga program`.

## 2. Load it onto the board

- **JTAG (Vivado):** set the PYNQ-Z2 boot jumper to JTAG (or keep SD; JTAG programming works either way), connect the micro-USB PROG/UART port, then `build_on_windows.ps1 -Program`, or Vivado > Hardware Manager > Program Device.
- **From the PYNQ Linux image (Jupyter):** copy `rvsoc_pynq_z2.bit` to the board and run `from pynq import Bitstream; Bitstream("rvsoc_pynq_z2.bit").download()`. The design is PL-only, so no `.hwh` is needed.

## 3. Board controls

| Board | Chip pin | Use |
|---|---|---|
| BTN0 | rst_n | Hold to reset, release to boot |
| BTN1 | ui[0] | Interrupt 0: prints `IRQ0` |
| BTN2 | ui[1] | Interrupt 1: prints `IRQ1` |
| BTN3 | ui[5] | GPIO input |
| SW0, SW1 | ui[2], ui[3] | Mode at reset: 00 demo, 01 GPIO mirror |
| LD0-LD3 | uo[2]-uo[5] | LED chaser (demo) / mirrored inputs (mirror mode) |

| Board | Chip pin | Use |
|---|---|---|
| LD4 | - | blue blink = alive, green = `Hello, TT!` seen |
| LD5 | - | green = `SELFTEST PASS`, red = `FAIL` seen |
| Pmod JA 1 | uo[0] UART TX | to the RX of a 3.3 V USB-UART adapter |
| Pmod JA 2 | ui[7] UART RX | from the TX of the adapter |
| Pmod JA 3 | uo[1] RTS | optional |
| Pmod JA 4 | ui[6] | GPIO input (pulled down) |
| Pmod JA 7-10 | flash CS#, SCK, SD0, PSRAM A CS# | logic analyser taps |
| Pmod JB 1-4, 7-10 | uo[0]..uo[7] | all chip outputs |
| Pmod GND | pins 5 and 11 | connect adapter GND |

The PYNQ-Z2's own USB-UART is wired to the Zynq ARM cores, not the FPGA fabric. That's why the chip's UART is on Pmod JA and needs an external 3.3 V USB-UART adapter (any CP2102/CH340/FT232 module). Without one, the status LEDs still show boot and self-test results.

## 4. Expected result

1. After programming: LD4 blinks blue.
2. Press and release BTN0: within about 10 ms LD4 turns green and LD5 turns green.
3. LD0..LD3 light one after another (100 ms per step). The light disappears for two steps while it is on uo[6]/uo[7], which go to Pmod JB only.
4. With a USB-UART at 115200 8N1 you see the same text as on the real chip (`docs/IO_GUIDE.md` section 2.2). Pressing BTN1 prints `IRQ0`.
5. Set SW0 = 1, SW1 = 0 and press BTN0: GPIO mirror mode. LD0 follows SW0 and LD1 follows SW1 (ui[2] -> uo[2], ui[3] -> uo[3]), LD3 follows BTN3.

## 5. Differences from the real chip

- The flash/PSRAM are emulated (64 KB each, same protocol and timing behaviour as the upstream TinyQV simulation model), with a fixed read latency of 2.
- The clock comes from the FPGA's MMCM, not from the TT demo board.
- The chip's I/O pads, the TT multiplexer and real QSPI board delays are not modelled; on silicon you may need read latency 2 or 3.